You may define a **viaQueues** method on your notification to indicate which queue should be utilized for each notification channel that the notification supports. An array of channel name / queue name pairs ought to be the result of this method:

```
public function viaQueues(): array
{
 return [
  ‘mail’ => ‘mail-queue’,
  ‘slack’ => ‘slack-queue’,
 ];
}
```

When queued alerts are sent within database transactions, the queue may handle them before the database transaction commits. Any adjustments you make to models or database records during the database transaction may not yet be reflected in the database if this occurs. Furthermore, any models or database entries produced during the transaction may no longer exist in the database. If your notification is based on these models, unanticipated problems may arise during the processing of the task that sends the queued notice.

If the **after_commit** configuration option for your queue connection is set to **false**, you may still specify that a specific queued notification be dispatched after all open database transactions have been committed by utilizing the **afterCommit** method when delivering the notification:

```
use App\Notifications\orderRecieved;
$user->notify((new orderRecieved($order))->afterCommit());
```

Or call the afterCommit method from constructor of notification.

```
<?php
namespace App\Notifications;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Notifications\Notification;
class orderRecieved extends Notification implements
ShouldQueue
{
 use Queueable;
```